

Automated Testing with Mocha - Simple Hinglish Summary

Source: <https://javascript.info/testing-mocha>

Main baat

Automated testing aage ke tasks me use hogi aur **real projects me bhi bahut common** hai.

1. Tests ki zarurat kyu?

Function likhte waqt hum jaante hain ki wo kya karna chahiye. Development me function ko console me chalakar result check karte hain, galat ho to fix karke dobara chalte hain.

Lekin manual re-run me kuch miss hona aasan hai.

Jaise function f banaya: $f(1)$ chal gaya, $f(2)$ nahi chala. Fix kiya to $f(2)$ chal gaya. Lekin $f(1)$ dobara check karna **bhool gaye**, aur wo toot gaya ho sakta hai. Ek cheez theek karte waqt dusri tod dena bahut common hai.

Automated testing: tests alag se, code ke saath likhe jate hain. Wo hamare functions ko alag alag tarike se chalte hain aur result ko expected se compare karte hain.

2. Behavior Driven Development (BDD)

BDD ek technique hai. **BDD = Tests + Documentation + Examples, teeno ek me.**

3. pow ka spec

Maan lo $\text{pow}(x, n)$ function banana hai jo x ko integer power n par le jaye ($n \geq 0$).

Code likhne se **pehle** hi describe karte hain ki function kya karega. Is description ko **specification (spec)** kehte hain. Isme use cases aur unke tests hote hain:

```
javascript

describe("pow", function() {

  it("raises to n-th power", function() {
    assert.equal(pow(2, 3), 8);
  });

});
```

Spec ke teen main building blocks:

Block	Kya karta hai
<code>describe("title", function() { ... })</code>	Kaunsi functionality describe ho rahi hai. <code>it</code> blocks ko group karta hai
<code>it("use case", function() { ... })</code>	Title me insaan ke padhne layak use case likhte hain, aur function us case ko test karta hai
<code>assert.equal(value1, value2)</code>	Function sahi ho to <code>it</code> ke andar ka code bina error ke chalna chahiye. <code>assert.equal</code> dono values compare karta hai, barabar na ho to error deta hai

4. Development ka flow

1. Sabse basic functionality ke tests ke saath **initial spec** likho.
2. **Initial implementation** banao.
3. **Mocha** framework se spec chalao. Functionality adhoori ho to errors dikhte hain. Sab theek hone tak corrections karo.
4. Ab initial implementation aur tests dono ready hain.
5. Spec me **aur use cases** add karo (jo implementation abhi support nahi karta). Tests fail hone lagte hain.
6. Step 3 par wapas jao, implementation update karo jab tak tests ke errors na jayein.
7. Functionality ready hone tak steps 3 se 6 repeat karo.

Development **iterative** hota hai: spec likho, implement karo, tests pass karo, aur tests likho, aur aage badho.

5. Kaunsi libraries use hoti hain

- **Mocha:** core framework. `describe`, `it` aur tests chalane wala main function deta hai.
- **Chai:** bahut saare **assertions** ki library. Abhi sirf `assert.equal` chahiye.
- **Sinon:** functions ko spy karne, built-in functions emulate karne ki library. Ye baad me chahiye hogi.

Ye browser aur server dono me kaam karti hain. Yahan browser wala variant hai.

Poora HTML page

```
html
```

```
<!DOCTYPE html>
<html>
<head>
  <!-- results dikhane ke liye mocha css -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/mocha/3.2
  <!-- mocha framework code -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/mocha/3.2.0/mocha.js"></s
  <script>
    mocha.setup('bdd'); // minimal setup
  </script>
  <!-- chai -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/chai/3.5.0/chai.js"></scr
  <script>
    // assert ko global bana do
    let assert = chai.assert;
  </script>
</head>

<body>

  <script>
    function pow(x, n) {
      /* function ka code abhi khali hai */
    }
  </script>

  <!-- tests wali script (describe, it...) -->
  <script src="test.js"></script>

  <!-- id="mocha" wala element results dikhayega -->
  <div id="mocha"></div>

  <!-- tests chalao! -->
  <script>
    mocha.run();
  </script>
</body>

</html>
```

Page ke 5 hisse:

1. `<head>`: third-party libraries aur tests ki styles
2. `<script>`: jis function ko test karna hai (yahan `pow`)

3. Tests: external script `test.js`
4. `<div id="mocha">`: Mocha yahan results dikhata hai
5. `mocha.run()`: tests shuru karta hai

Abhi test **fail** hoga kyunki `pow` khali hai aur `undefined` return karta hai.

(Zyada high-level test-runners bhi hain, jaise **karma**.)

6. Initial implementation

Pehle tests pass karne ke liye simple (cheating) implementation:

```
javascript

function pow(x, n) {
  return 8; // :) hum cheat kar rahe hain!
}
```

Test pass ho gaya. Lekin ye sahi function nahi hai, `pow(3, 4)` galat result dega. **Spec adhoora hai**, aur ye practice me bhi hota hai: tests pass, function galat.

7. Spec sudharna

`pow(3, 4) = 81` ka test add karte hain. Do tarike:

1. Ek hi `it` me do `assert`:

```
javascript

describe("pow", function() {

  it("raises to n-th power", function() {
    assert.equal(pow(2, 3), 8);
    assert.equal(pow(3, 4), 81);
  });

});
```

2. Do alag tests (behtar):

```
javascript
```

```
describe("pow", function() {

  it("2 raised to power 3 is 8", function() {
    assert.equal(pow(2, 3), 8);
  });

  it("3 raised to power 4 is 81", function() {
    assert.equal(pow(3, 4), 81);
  });

});
```

Fark: `assert` me error aane par `it` block **turant ruk jata hai**. Pehle variant me pehla `assert` fail ho to dusre ka result kabhi nahi dikhega. Alag tests se **zyada jaankari** milti hai, isliye doosra variant behtar hai.

Ek zaruri rule: Ek test, ek cheez check kare. Test me do independent checks dikhein to use do saral tests me tod do.

Ab dusra test fail hoga kyunki function hamesha `8` deta hai.

8. Implementation sudharna

```
javascript

function pow(x, n) {
  let result = 1;

  for (let i = 0; i < n; i++) {
    result *= x;
  }

  return result;
}
```

Aur values par test karne ke liye `it` blocks haath se likhne ki jagah `for` se generate kar sakte ho:

```
javascript
```

```
describe("pow", function() {

  function makeTest(x) {
    let expected = x * x * x;
    it(`${x} in the power 3 is ${expected}`, function() {
      assert.equal(pow(x, 3), expected);
    });
  }

  for (let x = 1; x <= 5; x++) {
    makeTest(x);
  }

});
```

9. Nested `describe`

`makeTest` aur `for` ka kaam ek hi hai (given power tak check karna), isliye unhe **nested** `describe` se group karte hain:

javascript

```
describe("pow", function() {

  describe("raises x to power 3", function() {

    function makeTest(x) {
      let expected = x * x * x;
      it(`${x} in the power 3 is ${expected}`, function() {
        assert.equal(pow(x, 3), expected);
      });
    }

    for (let x = 1; x <= 5; x++) {
      makeTest(x);
    }

  });

  // ... aur tests yahan (describe aur it dono add kar sakte hain)
});
```

Nested `describe` ek naya "subgroup" banata hai. Output me indent ke saath title dikhta hai. Top level par aur `it` aur `describe` add karne par unhe `makeTest` nahi dikhega.

`before/after` aur `beforeEach/afterEach`

Tests se pehle/baad chalne wale functions:

javascript

```
describe("test", function() {

  before(() => alert("Testing started - before all tests"));
  after(() => alert("Testing finished - after all tests"));

  beforeEach(() => alert("Before a test - enter a test"));
  afterEach(() => alert("After a test - exit a test"));

  it('test 1', () => alert(1));
  it('test 2', () => alert(2));

});
```

Chalne ka order:

```
Testing started - before all tests (before)
Before a test - enter a test (beforeEach)
1
After a test - exit a test (afterEach)
Before a test - enter a test (beforeEach)
2
After a test - exit a test (afterEach)
Testing finished - after all tests (after)
```

Ye aam taur par **initialization, counters zero karne** jaise kaamon ke liye use hote hain.

10. Spec badhana

`pow(x, n)` sirf positive integer `(n)` ke liye hai. Galat `(n)` par JS functions aam taur par `NaN` return karte hain.

Pehle spec me behavior likho(!):

javascript

```
describe("pow", function() {

    // ...

    it("for negative n the result is NaN", function() {
        assert.isNaN(pow(2, -1));
    });

    it("for non-integer n the result is NaN", function() {
        assert.isNaN(pow(2, 1.5));
    });

});
```

Naye tests fail honge kyunki implementation abhi support nahi karta. **BDD ka tarika yahi hai:** pehle failing tests likho, phir unke liye implementation banao.

Aur Chai assertions:

Assertion	Kya check karta hai
<code>assert.equal(v1, v2)</code>	<code>v1 == v2</code>
<code>assert.strictEqual(v1, v2)</code>	<code>v1 === v2</code>
<code>assert.notEqual</code> , <code>assert.notStrictEqual</code>	Upar wale ka ulta
<code>assert.isTrue(value)</code>	<code>value === true</code>
<code>assert.isFalse(value)</code>	<code>value === false</code>
<code>assert.isNaN(value)</code>	Value <code>NaN</code> hai

Ab implementation me 2 lines add karo:

```
javascript
```

```
function pow(x, n) {  
  if (n < 0) return NaN;  
  if (Math.round(n) !== n) return NaN;  
  
  let result = 1;  
  
  for (let i = 0; i < n; i++) {  
    result *= x;  
  }  
  
  return result;  
}
```

Ab saare tests pass ho jate hain.

Summary

BDD me **pehle spec, phir implementation**. End me spec aur code dono milte hain.

Spec teen tarike se kaam aata hai:

1. **Tests:** guarantee dete hain ki code sahi chalta hai.
2. **Docs:** `describe` aur `it` ke titles batate hain ki function kya karta hai.
3. **Examples:** tests asal me **working examples** hain ki function kaise use karna hai.

Spec ke saath function ko **safely improve, change, ya poora dobara likh** sakte hain aur pakka kar sakte hain ki wo abhi bhi sahi chal raha hai.

Bade projects me ye bahut zaruri hai, jab function kai jagah use hota ho. Function badalne par har jagah haath se check karna namumkin hai.

Tests ke bina do raaste hote hain:

1. Badlav kar do chahe kuch bhi ho, aur users ko bugs milein.
2. Ya galti ki saza kadi ho to log function chhune se **darne** lagte hain, aur code purana ho jata hai.

Automatic testing in problems se bachata hai. Badlav ke baad tests chalao aur kuch hi seconds me kai checks ho jate hain.

Ek aur fayda: achhe-tested code ka **architecture behtar** hota hai. Test likhne ke liye har function ka task saaf aur input/output tay hona chahiye.

Asli zindagi me spec pehle likhna kabhi mushkil hota hai, kyunki pata nahi hota code kaise behave karega. Lekin aam taur par tests likhne se development **tez aur stable** hoti hai.

Abhi tests likhna zaruri nahi hai, lekin unhe **padhna** aana chahiye.

Practice Task (Answer)

Is test me kya galat hai?

javascript

```
it("Raises x to the power n", function() {
  let x = 5;

  let result = x;
  assert.equal(pow(x, 1), result);

  result *= x;
  assert.equal(pow(x, 2), result);

  result *= x;
  assert.equal(pow(x, 3), result);
});
```

Jawab: Ye asal me **3 tests hain**, jo ek function me 3 asserts ki tarah likhe gaye hain. Error aaye to samajhna mushkil hota hai ki kya galat gaya, hume test hi **debug** karna padta hai.

Behtar: kai `it` blocks me todo, jisme input aur output saaf likhe hon:

javascript

```
describe("Raises x to power n", function() {
  it("5 in the power of 1 equals 5", function() {
    assert.equal(pow(5, 1), 5);
  });

  it("5 in the power of 2 equals 25", function() {
    assert.equal(pow(5, 2), 25);
  });

  it("5 in the power of 3 equals 125", function() {
    assert.equal(pow(5, 3), 125);
  });
});
```

Tip: `it.only` se ek akela test alag se chala sakte ho:

javascript

```
it.only("5 in the power of 2 equals 25", function() {  
  assert.equal(pow(5, 2), 25);  
});
```

Quick Summary

Baat	Yaad rakho
BDD	Tests + Docs + Examples
<code>describe</code>	Tests ka group
<code>it</code>	Ek use case ka test
<code>assert.*</code>	Result check karta hai
Rule	Ek test = ek cheez
Flow	Spec → Implementation → Tests pass → Aur spec
<code>beforeEach/afterEach</code>	Har test ke pehle/baad chalte hain
<code>it.only</code>	Sirf ek test chalata hai